NEOSTATS

Intelligence. Innovation. Impact.

Candidate Assignment

AI Engineer Role

AI-Powered Credit Risk Intelligence Platform

DATASET LINK: *https://www.kaggle.com/competitions/home-credit-default-risk/data*

1. Objective

Design and build a lightweight AI-powered credit risk platform using the Home Credit Default Risk dataset. This assignment tests your ability to work across the full AI engineering stack — from data analysis to a deployed, explainable application.

Your submission must include:

- Exploratory data analysis and key insights
- A machine learning model to predict loan default
- Explainable AI to interpret predictions
- A conversational talk-to-data interface
- Business-readable decision rules derived from ML
- A simple user interface
- Dockerized deployment with setup instructions

2. Business Context

Banks face pressure to make faster, more accurate, and explainable credit decisions. This platform addresses real banking needs:

- Identify high-risk loan applicants early
- Automate risk scoring and decision support
- Provide explainable reasons for risk classifications
- Satisfy audit and regulatory requirements
- Let business analysts explore data in plain English
- Bridge ML insights and credit policy with rules

3. Assignment Scope

#	Module	What to Build
1	Data Understanding & EDA	Analyse the dataset — demographics, financials, credit history, repayment behaviour, and data quality.
2	Talk-to-Data System	Let users ask natural-language questions about the data and get readable answers backed by SQL queries.
3	Machine Learning Layer	Train a model to predict default probability. Output a risk score and risk band (Low / Medium / High).
4	Explainable AI	Use SHAP or LIME to explain each prediction. Show which features drove the risk classification.
5	User Interface	Build a multi-section UI covering EDA, risk prediction, explainability, rules, and the chatbot.
6	Dockerized Deployment	Containerize the app with a Dockerfile and docker-compose.yml. It must run with minimal setup.

4. Functional Requirements

Part 1: Data Understanding & Exploratory Analysis

Understand the dataset before building any model. Your analysis should cover the following areas:

Deliverables:

Dataset summary, data quality observations, feature categorization, at least 5 business insights, and supporting charts/visualizations.

Part 2: Talk-to-Data System

Build a chatbot that converts natural-language questions into SQL queries, runs them, and returns readable business insights.

Part 3: Machine Learning Layer

Train a model to predict the probability that a loan applicant will default. Focus on a clean pipeline, proper evaluation, and handling class imbalance.

Part 4: Explainable AI

Apply at least one explainability method to make each prediction understandable to a non-technical user.

Part 5: User Interface

Build a lightweight multi-section UI to demonstrate the full platform end to end.

Part 6: Dockerized Deployment

Containerize the application so an evaluator can run it with a single command.

Required files:

- Dockerfile — define environment and dependencies
- docker-compose.yml — orchestrate all services
- .env.example — document all required environment variables

The evaluator should be able to clone your repo, run docker-compose up, and have the full application

5. Recommended Technical Stack

Use the stack below. If you choose something different, briefly explain your reasoning in the README.

Component	Recommended Choice
LLM	Any LLM model of your choice
Deployment	Docker + Docker Compose

6. Expected Deliverables

Code (in your Github Repo repository):

- Data processing and feature engineering pipeline
- ML training and inference pipeline with saved model artifacts
- Talk-to-data module with prompt templates and SQL validation
- Rule derivation module
- UI application code
- Dockerfile and docker-compose.yml in the repo
- README.md with setup instructions

Documentation (in README.md):

- Architecture overview or component diagram
- Step-by-step setup and run instructions
- Model selection rationale and class imbalance strategy
- Evaluation metrics and results
- Prompt engineering and token optimization approach
- Rule derivation logic and sample outputs
- Known limitations and possible improvements

8. Evaluation Criteria

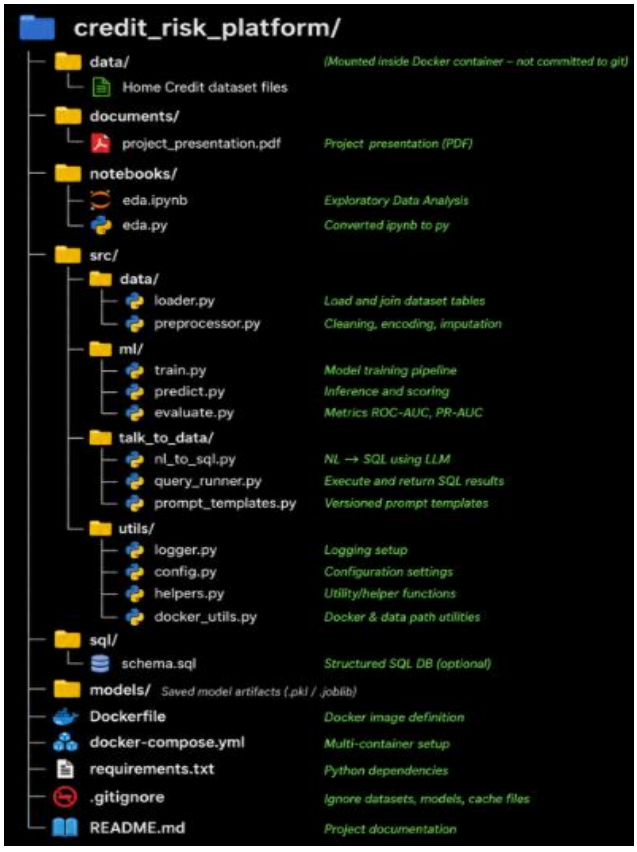
Your submission will be scored across the following areas. A strong submission shows depth in all of them, not just the ML model.

Evaluation Area	Weightage
ML solution design and model quality	30%
LLM integration and talk-to-data capability	25%
Dockerization and engineering quality	10%
EDA	15%
Prompt optimization and hallucination control	15%
Documentation and architecture clarity	5%

9. What We Look For

- Sensible preprocessing and handling of class imbalance
- Reliable NL – SQL agent chatbot with at least 5 working query
- Clean, modular, readable code
- End-to-end workflow
- README file (well-explained solutions over complex, hard-to-run ones.)

Code Structure to be followed:



10. Submission Instructions

- GitHub repository link or zipped source code (no virtual environment folders)
- README with step-by-step setup and run instructions
- .env.example with all required environment variables
- Brief note on major design decisions
- Make a Presentation on the Use case with output screenshots and keep the PPT as pdf file inside the code folder(/documents folder) and then submit it.
- A Microsoft Form will be shared for final submission.